

Modèle de données

Application web **Express Voitures** : gestion du stock d'une concession de voitures d'occasion

Projet 5 · ASP.NET Core MVC · Entity Framework Core (Code First) · SQL Server | Auteur : Dominique | 06/2026

Objet du document. Ce document présente le modèle de données du prototype demandé par Jacques : le *schéma relationnel*, le *diagramme UML* des entités, le détail de chaque table, puis les *choix de conception* et leurs justifications. Les entités seront créées en C# (approche *Code First*) et la base générée par EF Core dans SQL Server.

Pourquoi cette démarche ? La conception orientée métier (DDD)

Avant d'écrire la moindre ligne de code ou de créer la moindre table, on commence par **modéliser le métier** : comprendre la réalité de l'activité de Jacques (acheter des voitures, les réparer, les revendre) et la traduire en *entités* et en *règles*. C'est tout l'objet de ce document.

Cette priorité donnée au métier porte un nom : le **DDD**.

Le **DDD** (en C#) signifie **Domain-Driven Design**, ou **conception pilotée par le domaine**. C'est une approche de conception logicielle qui **met le métier au centre du code**, plutôt que la technique ou la base de données.

Concrètement, dans ce projet, cela se traduit ainsi :

- les **entités** (Marque, Modèle, Voiture, Vente, Reparation, TypeReparation) reflètent des concepts *réels* de la concession, et non des contraintes techniques ;
- les **règles** proviennent du métier : prix de vente = prix d'achat + réparations + 500 €, année comprise entre 1990 et l'année courante, statut déduit des dates ;
- la **base de données n'est pas le point de départ, mais une conséquence** du modèle : avec l'approche *Code First* d'EF Core, on écrit d'abord les classes du domaine, puis EF Core génère la base à partir d'elles.

L'intérêt : un code qui « parle le langage du métier », plus facile à faire évoluer et surtout à **expliquer à Jacques** (l'enjeu de la maintenance). On distingue d'ailleurs deux couches complémentaires : le **Domaine** (les entités, le métier) et l'**Infrastructure** (ce qui dialogue avec la base de données, ici EF Core).

À nuancer. Pour ce prototype, on retient surtout *l'esprit* du DDD (le métier d'abord) sans nécessairement déployer tous ses patrons avancés (agrégats, objets-valeur, dépôts...), l'énoncé restant volontairement simple. L'essentiel est que le modèle de données découle de la réalité de Jacques, pas de l'inverse.

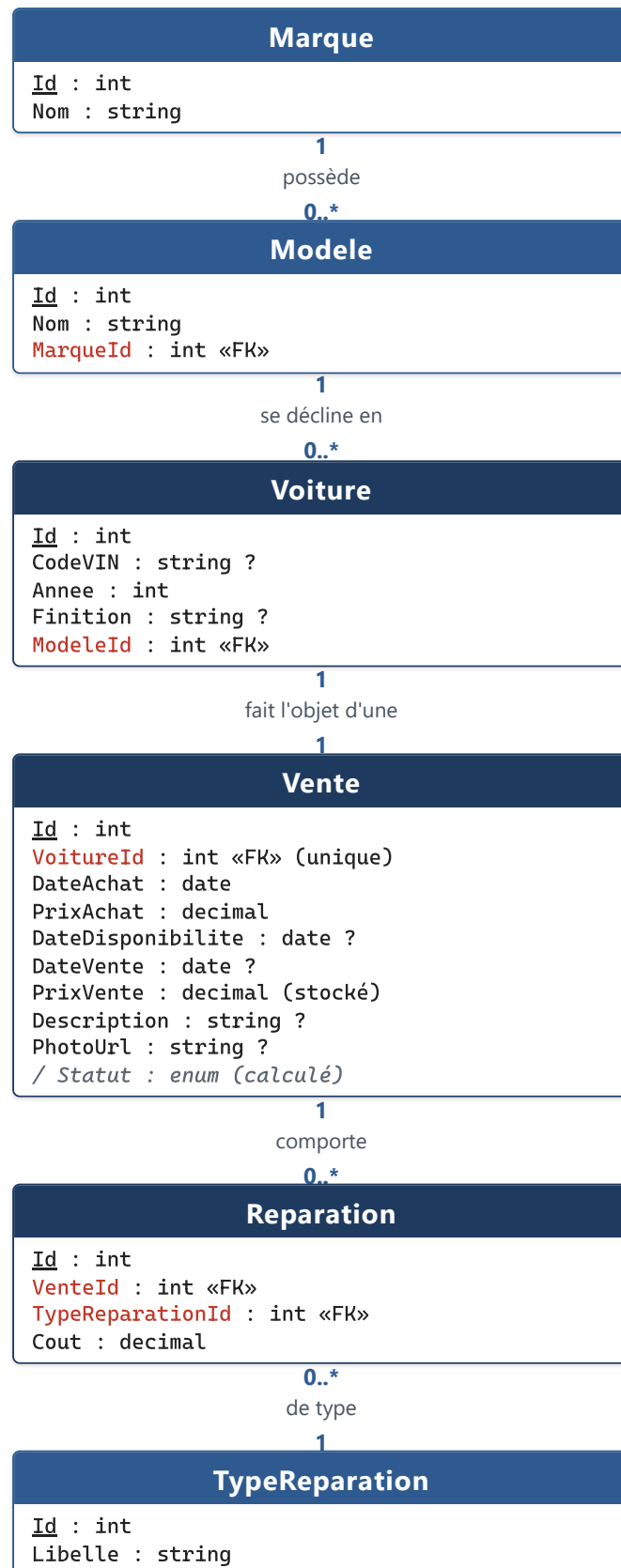
1. Schéma relationnel



Le cœur du modèle est une chaîne de relations **un-à-plusieurs** : une **marque** regroupe zéro, un ou plusieurs **modèles** ; un **modèle** se décline en zéro, une ou plusieurs **voitures** ; une **vente** comporte zéro, une ou plusieurs **réparations**. Entre **Voiture** et **Vente**, la relation est **un-à-un** : une voiture donne lieu à exactement une vente (un cycle d'achat-revente), et inversement. Enfin, chaque **réparation** est rattachée à un **type de réparation** du référentiel (réutilisable d'une voiture à l'autre).

Le symbole $\text{---}<$ (« patte-d'oie ») se lit : côté simple = 1 (obligatoire, garanti par la clé étrangère), côté éclaté = 0..* (zéro, un ou plusieurs). Les entités en bleu clair (**Marque**, **Modele**, **TypeReparation**) sont des **référentiels** : des listes de valeurs réutilisables.

2. Diagramme UML des entités



souligné = clé primaire · «FK» = clé étrangère · « ? » = optionnel · / *italique* = propriété calculée

3. Détail des entités

Marque (référentiel)

Attribut	Type	Obligatoire	Notes
Id	int	clé primaire	auto-incrément
Nom	texte	oui	unique

Modele (référentiel)

Attribut	Type	Obligatoire	Notes
Id	int	clé primaire	auto-incrément
Nom	texte	oui	
MarqueId	int (FK Marque)	oui	

Contrainte d'unicité composée : **(Marqueld, Nom)**

Voiture (identité physique)

Attribut	Type	Obligatoire	Notes
Id	int	clé primaire	clé technique
CodeVIN	texte	optionnel	unique si renseigné
Annee	int	oui	$1990 \leq x \leq \text{année courante}$
Finition	texte	optionnel	ex. LE, Sport, XLT
ModeleId	int (FK Modele)	oui	porte la marque (via Modele)

Vente (volet prix, dates et présentation)

Attribut	Type	Obligatoire	Notes
Id	int	clé primaire	
VoitureId	int (FK Voiture)	oui	unique (relation 1 à 1)
DateAchat	date	oui	événement de création
PrixAchat	décimal	oui	≥ 0
DateDisponibilite	date	optionnel	null tant qu'en préparation
DateVente	date	optionnel	null tant que non vendue
PrixVente	décimal	oui (stocké)	= PrixAchat + somme des Cout + 500 ; figé à la vente
Description	texte	optionnel	ajoutée plus tard
PhotoUrl	texte	optionnel	ajoutée plus tard
Statut	énum.	calculé	déduit des dates

TypeReparation (référentiel)

Attribut	Type	Obligatoire	Notes
Id	int	clé primaire	auto-incrément
Libelle	texte	oui	unique (Pneus, Plaquettes, Freins, Climatisation...)

Reparation (association porteuse de coût)

Attribut	Type	Obligatoire	Notes
Id	int	clé primaire	auto-incrément
VenteId	int (FK Vente)	oui	
TypeReparationId	int (FK TypeReparation)	oui	remplace l'ancien libellé libre
Cout	décimal	oui	≥ 0

Prix de vente = Prix d'achat + Somme des coûts de réparation + 500 € (règle métier de Jacques : vente en volume)

4. Choix de conception et justifications

4.1. Clé primaire technique (Id) plutôt que le CodeVIN

On distingue la **clé naturelle** (donnée métier réutilisée comme identifiant, ici le VIN) de la **clé de substitution** (identifiant technique sans signification, ici un Id entier). On retient la clé de substitution car :

- **Une clé primaire doit toujours exister** : le VIN est souvent inconnu à la saisie (la colonne *Code VIN* du tableur de Jacques est vide) ; or une clé primaire ne peut être nulle.
- **Une clé primaire ne doit jamais changer** : un VIN de 17 caractères peut comporter une faute et devoir être corrigé ; s'il sert de clé, il faudrait répercuter la correction sur toutes les références.
- **Robustesse** : un VIN peut être dupliqué (clonage de véhicule), erroné ou absent ; l'identité interne ne doit pas reposer sur une donnée aussi fragile.
- **Convention EF Core** : une propriété Id est reconnue automatiquement comme clé primaire et générée en auto-incrément (**IDENTITY**).
- **Performance** : un entier (4 octets) donne des index et des jointures plus légers qu'une chaîne de 17 caractères.

Le CodeVIN reste donc un simple champ d'information **optionnel**, avec un index unique facultatif pour interdire les doublons saisis.

4.2. ModèleId dans Voiture (et marque non dupliquée)

La **marque est déjà connue par le modèle** (chaque **Modèle** porte son **MarqueId**). Depuis une voiture, on atteint donc la marque par navigation : **Voiture** → **Modèle** → **Marque**. Ajouter en plus un champ marque sur **Voiture** serait une **redondance** ouvrant la porte à une **incohérence** (ex. une « Civic » rattachée à « Ford »). On élimine ainsi une **dépendance transitive** (3^e forme normale) : un attribut non-clé ne doit pas dépendre d'un autre attribut non-clé. La marque appartient au *modèle*, pas à la *voiture*.

4.3. Unicité d'un modèle sur le couple (MarqueId, Nom)

Un nom de modèle **n'est pas unique au niveau mondial** : le badge « GTI » a été porté par Volkswagen (Golf) comme par Peugeot (205), entre autres. Ce sont deux modèles **distincts** qui partagent un nom : un nom partagé n'est pas une entité partagée. On ne crée donc pas de relation plusieurs-à-plusieurs ; on enregistre deux lignes **Modèle** séparées, et l'unicité porte sur le **couple (marque, nom)** et non sur le nom seul. C'est précisément la marque (via **ModèleId**) qui lève l'ambiguïté.

4.4. Séparation Voiture / Vente (le produit et le financier)

On sépare ce qui **définit la voiture** (son identité physique : VIN, année, modèle, finition) de son **volet commercial et financier** (prix, dates, réparations, présentation), regroupé dans l'entité **Vente**. C'est la démarche suggérée dès l'analyse (l'exemple « fruits / légumes » : une entité *produit*, une entité *financière*). Avantages : des **responsabilités séparées** (un principe SOLID), des entités plus lisibles, et

une évolution facilitée. La relation est de type **1 à 1** : une voiture donne lieu à exactement une vente, et réciproquement.

4.5. Prix de vente stocké (et non calculé)

Le prix de vente résulte de la règle métier (**PrixAchat + somme des coûts de réparation + 500 €**). On pourrait le recalculer à la volée, mais on choisit de le **stocker**, pour deux raisons :

- une fois la voiture **vendue**, ce prix devient un **fait comptable historique**, figé : il ne doit plus jamais bouger, même si d'autres données venaient à évoluer ;
- il est ainsi **directement extractible pour la comptabilité**, sans recalcul.

Mise en œuvre : la valeur est **recalculée automatiquement** à chaque enregistrement tant que la voiture est invendue, puis **figée** dès que **DateVente** est renseignée. C'est une **dénormalisation assumée et documentée**, justifiée par le caractère historique de la donnée (le prix réel de la transaction n'est plus une simple valeur déductible).

4.6. Statut calculé à partir des dates

L'état d'une voiture se déduit des dates (portées par **Vente**), sans champ supplémentaire à maintenir :

- **En préparation** : **DateDisponibilite** est nulle ;
- **Disponible à la vente** : **DateDisponibilite** renseignée et **DateVente** nulle ;
- **Vendue** (plus disponible) : **DateVente** renseignée.

« Marquer une voiture comme plus disponible » revient donc à renseigner sa **DateVente**.

4.7. Référentiel TypeReparation et Reparation en association

Les réparations récurrentes (Pneus, Plaquettes, Freins, Climatisation...) sont regroupées dans un référentiel **TypeReparation**, **réutilisable** d'une voiture à l'autre, plutôt qu'un libellé libre « fourre-tout » ressaisi à chaque fois (source d'incohérences : « Pneus », « pneu », « pneus avant »...).

Reparation devient alors une **entité d'association** : elle relie une **Vente** à un **TypeReparation** et porte le **Cout** de l'intervention. La liste « Pneus, freins » d'une cellule du tableur se traduit ainsi par **plusieurs lignes Reparation**, chacune avec son type et son coût, dont la **somme alimente le prix de vente**.

4.8. Nullabilité des champs et cycle de vie

Les champs reflètent les étapes de vie d'une voiture : **Achat → Réparations → Disponibilité → Vente**. **DateAchat**, **PrixAchat** et **Annee** sont obligatoires (connus dès la création) ; **DateDisponibilite**, **DateVente**, **CodeVIN**, **Finition**, **Description** et **PhotoUrl** sont optionnels (renseignés ou ajoutés ultérieurement). L'**Annee** est validée entre **1990 et l'année courante**, conformément à l'exigence de cohérence des saisies de Jacques.

4.9. Cardinalités : la relation 1 à 1 et le 0 .. * du côté « plusieurs »

Une relation a **deux extrémités qui ne sont pas symétriques**. Du côté « enfant » (le 1), la clé étrangère impose que chaque modèle ait une marque, chaque voiture un modèle, chaque réparation une vente et un type. Du côté « parent » (le 0 .. *), la clé étrangère **ne dit rien** sur le nombre d'enfants :

structurellement, une marque peut exister sans aucun modèle, et une vente sans aucune réparation (voiture achetée en bon état). Une clé étrangère contraint toujours l'*enfant*, jamais le *parent* ; d'où le $0 \dots *$ (et non $1 \dots *$).

Le cas particulier **Voiture / Vente** est une relation **1 à 1** : chaque voiture a exactement une vente, et chaque vente concerne exactement une voiture. Techniquement, c'est **Vente** qui porte la clé étrangère **VoitureId**, assortie d'une **contrainte d'unicité** (pour empêcher deux ventes de pointer la même voiture).

Sécurité et rôles (rappel pour la suite). La consultation est ouverte à tous ; seules les opérations d'ajout et de modification seront réservées au gérant. Les tables d'utilisateurs et de rôles seront ajoutées automatiquement par *ASP.NET Core Identity* (« Individual Accounts ») : elles ne figurent pas dans ce modèle métier.